Nil Guards are also used quite often to initialize instance variables. Look at this class:

```
class C
  def initialize
    @a = []
  end

  def elements
    @a
  end
end
```

By using a Nil Guard, you can rewrite the same code more succinctly:

```
class C
  def elements
    @a ||= []
  end
end
```

The previous code initializes the instance variable at the latest possible moment, when it's actually accessed. This idiom is called a *Lazy Instance Variable*. Sometimes, as in the earlier example, you manage to replace the whole initialize method with one or more Lazy Instance Variables.

Spell: Lazy Instance Variable

Nil Guards and Boolean Values

Nil Guards have one quirk that is worth mentioning: they don't work well with Boolean values. Here is an example:

```
def calculate_initial_value
  puts "called: calculate_initial_value"
  false
end

b = nil
2.times do
  b ||= calculate_initial_value
end
```

```
called: calculate_initial_value
called: calculate_initial_value
```

The Nil Guard in the code above doesn't seem to work—calculate_initial_value is called twice, instead of once as you might expect. To see where the problem is, let's write the if equivalent of that Nil Guard.